MA203 - Numerical Methods
Lecture Notes

Abhinav Jha
Indian Institute of Technology, Gandhinagar
Winter Semester 2026

Preface

These notes are designed to provide a structured and comprehensive understanding of the course content. They will cover key topics, concepts, and computational techniques that are fundamental to numerical methods. Please note that this is the first iteration (Version 0.0.1) of the notes and hence there is a chance that some of the content is incorrect. If you find some flaws, please email me at `abhinav.jha@iitgn.ac.in`.

Obstacles are Opportunities.

—Peacemaker

Contents

1 Floating Point Arithmetic **7**
1.1 Binary and Decimal Number System 9
1.1.1 Normalized Scientific Notation 11
1.1.2 IEEE Notation 11
1.2 Errors 13
1.2.1 Measure of Error 13
1.2.2 Arithmetic Errors 14

List of Mathematical Notations **19**

Chapter 1

Floating Point Arithmetic

In engineering sciences, there will be times when one has to solve equations, interpolate data, or compute certain quantities of interest. Finding the (exact) solution of an equation (or any of the aforementioned tasks) is, in principle, the job of mathematics. However, whenever a mathematician sees an equation, instead of directly solving it, we first ask two very important questions:

1. **Existence**: Does a solution exist?
2. **Uniqueness**: If a solution exists, is it unique?

Let us look at two very simple examples:
Example 1:

$$\sum_{n=1}^{\infty} \frac{1}{n}$$

and
Example 2:

$$\sum_{n=1}^{\infty} \frac{1}{n^2}.$$

If you ask any undergraduate student who has studied Calculus, they would say that the first series diverges, whereas the second series converges. This is due to the fact that a p-series, that is,

$$\sum_{n=1}^{\infty} \frac{1}{n^p},$$

converges for $p \in \mathbb{Z}$ with $p > 1$. The idea of convergence of a series was introduced by the French mathematician Baron Augustin-Louis Cauchy (fun fact: his name is inscribed on the Eiffel Tower).

Most areas of mathematics are content with the notions of existence and uniqueness of a solution, but not all. See, even if we have existence and uniqueness, the real challenge is to actually *compute* the solution. In certain cases, we do not even have proofs of existence and uniqueness, but we still need the solution. A classic example of this is the three-dimensional

Figure 1.1: Baron Augustin-Louis Cauchy (21 August 1789 – 23 May 1857).

Navier–Stokes equations. In fact, if you prove existence and uniqueness for this problem, you can win a prize of one million dollars. But if you work in Computational Fluid Dynamics, you already know how important these equations are.

Now, analytical solutions do not always exist, and even if they do, obtaining them in closed form can be extremely difficult. For example, the solution of an ordinary differential equation (ODE) can sometimes be expressed using a power series, but since such series are infinite, implementing them in practice is tough. Also, humans are impatient by nature, and we prefer to delegate the job of computing solutions to computers.

Let us look at Example 2 again and compute the error and the time taken to evaluate the partial sum for different values of n, as shown in Table 1.1.

n	Approximate Sum	Error	Time (s)
10^1	1.54976773	9.52×10^{-2}	0.0000
10^2	1.63498390	9.95×10^{-3}	0.0000
10^3	1.64393457	1.00×10^{-3}	0.0002
10^4	1.64483407	1.00×10^{-4}	0.0024
10^5	1.64492407	1.00×10^{-5}	0.0271
10^6	1.64493307	1.00×10^{-6}	0.2756
10^7	1.64493397	1.00×10^{-7}	2.8460

Table 1.1: Convergence of the series $\sum_{k=1}^{n} \frac{1}{k^2}$ towards $\pi^2/6$.

The true solution of this problem is

$$\frac{\pi^2}{6},$$

which one obtains using Fourier analysis, where this problem is known as the Basel problem. If we look at Table 1.1 carefully, we notice that we obtain a reasonably good approximation of the true solution within seconds. This is the power of numerical methods. However, also notice that after a certain value of n, the reduction in error is not very significant, while the time required increases drastically.

From now until the end of your engineering life, whenever you solve a mathematical problem, there is a 99% chance that you will need to use a numerical method.

I would like to make a clear distinction between two topics: *Numerical Methods* and *Numerical Analysis*. The development of a numerical method may sometimes lead to a false

sense of confidence. There can be situations where a method works for 9 out of 10 problems, but fails for the 10th one. Such methods can be dangerous.

A very infamous example of this is the Tacoma Narrows Bridge collapse of 1940. The bridge collapsed within four months of its opening, the primary reason being a lack of proper structural and eigenvalue analysis during the design phase.

Figure 1.2: Collapse of the Tacoma Narrows Bridge.

Numerical analysis is the branch of mathematics that studies these methods, analyzes their failure rates through error analysis, and tries to make them computationally efficient. This course will only lightly touch upon ideas from numerical analysis and will not go into great depth.

The topic *Numerical Methods* has the word "numerical" in it, which means that we want to solve equations numerically, that is, using calculators or computers. The numbers you have worked with until now belong to the decimal number system. However, computers work using the binary number system. As a result, whenever we solve a problem on a computer, numbers must be converted from one system to another, which inevitably introduces errors.

That is why studying errors is important. In this chapter, we will deal with number systems and the different kinds of errors that we encounter on computers.

1.1 Binary and Decimal Number System

In mathematics, we usually work with the *decimal number system*, also called the *base* 10 *system*. In this system, numbers are written as powers of 10. For example,

$$619.916 = 6 \times 10^2 + 1 \times 10^1 + 9 \times 10^0 + 9 \times 10^{-1} + 1 \times 10^{-2} + 6 \times 10^{-3}.$$

Since everything is expressed in powers of 10, we call this the base 10 number system. If we allow infinitely many digits to the right of the decimal point, then we can represent irrational numbers such as π or e.

Computers, however, do not work with the decimal number system. Instead, they use the *binary number system*, also known as the *base* 2 *system*, which consists of only two digits: 0 and 1. In this system, numbers are written as powers of 2.

Let us briefly discuss how to convert numbers from the decimal system to the binary system and vice versa.

Conversion from decimal to binary

To convert a *whole number* from decimal to binary, we repeatedly divide the number by 2 and record the remainders. The binary representation is obtained by reading the remainders from bottom to top.

Example 1.1. Let us take

$$x = 25_{10},$$

where the subscript $_{10}$ indicates that the number is written in base 10.
We perform successive division by 2:

Division	Quotient	Remainder
25/2	12	1
12/2	6	0
6/2	3	0
3/2	1	1
1/2	0	1

Reading the remainders from bottom to top, we obtain

$$25_{10} = 11001_2.$$

To verify this, we write the binary number using powers of 2:

$$11001_2 = 1 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 = 16 + 8 + 1 = 25.$$

Binary representation of fractional numbers

For numbers less than 1, we use negative powers of 2. To convert a decimal fraction to binary, we repeatedly multiply the number by 2 and record the integer part at each step.

Example 1.2. Let us convert

$$0.125_{10}$$

to binary.

Multiplication	Result	Integer Part
0.125×2	0.25	0
0.25×2	0.50	0
0.50×2	1.00	1

Reading the integer parts from top to bottom, we obtain

$$0.125_{10} = 0.001_2.$$

Combining both parts, we conclude that

$$25.125_{10} = 11001.001_2.$$

This example already hints at an important issue: while some decimal numbers have a finite binary representation, many do not. This fact plays a crucial role in understanding numerical errors on computers, which we will study next.

1.1.1 Normalized Scientific Notation

The numbers we work with can quickly become very large or very small, making them inconvenient to write and store. To deal with this, we use *scientific notation*. In the decimal number system, any non-zero real number can be written in the form

$$x_{10} = \pm r \times 10^n,$$

where

$$0.1 \leq r < 1 \quad \text{and} \quad n \in \mathbb{Z}.$$

If $x = 0$, then we simply take $r = 0$.

Example 1.3. The number

$$1301.2022$$

can be written in scientific notation as

$$1301.2022 = 0.13012022 \times 10^4.$$

In exactly the same way, we can use scientific notation in the binary number system. A non-zero binary number can be written as

$$x_2 = \pm q_2 \times 2^m,$$

where

$$0.1_2 \leq q_2 < 1.0_2 \quad \text{and} \quad m \in \mathbb{Z}.$$

If $x_2 = 0$, then we take $q_2 = 0$.

In both the decimal and binary scientific notations, the quantity r (or q_2) is called the *mantissa*, and the power n (or m) is called the *exponent*.

1.1.2 IEEE Notation

To store binary numbers in a computer, IEEE published a unified framework in 1985 [1], and later released an updated version in 2008 [2]. This standard defines formats for storing numbers in single, double, and extended precision, and is followed by virtually all modern computers.

In the double-precision format, a total of 64 bits are used to represent a real number. The first bit is the *sign bit*, denoted by s, which is 0 for positive numbers and 1 for negative numbers. This is followed by an 11-bit exponent c, called the *characteristic*, and a 52-bit binary fraction f, called the *mantissa*. The base of the exponent is 2.

A schematic representation of this storage format is shown in Fig. 1.3.

Now, we have allocated 52 bits for the mantissa, and hence it can represent numbers up to a precision of 2^{-52}. If we express this precision in the decimal number system, it corresponds to a quantity of the form 10^{-d}. Therefore,

$$10^{-d} = 2^{-52}.$$

Sign	Exponent	Mantissa
s	c	f
1 bit	11 bits	52 bits

Figure 1.3: Visual representation of the IEEE double-precision (64-bit) floating-point storage format.

Taking $\log_{10}$ on both sides, we obtain

$$\begin{aligned} \log_{10}(10^{-d}) &= \log_{10}(2^{-52}), \\ -d &= -52\log_{10}(2), \\ d &\approx 15.65 \approx 16. \end{aligned}$$

Thus, a double-precision floating-point number can represent approximately 16 decimal digits accurately.

The exponent field consists of 11 bits, which allows for $2^{11} = 2048$ possible values. However, not all of these values are used to represent actual exponents. To allow both positive and negative exponents, the exponent is stored in a *biased* form. In double precision, the bias is 1023.

As a result, the actual exponent ranges approximately from -1023 to 1024. With this convention, a normalized floating-point number is represented as

$$(-1)^s 2^{c-1023}(1+f),$$

where

$$0 \leq c \leq 2047,$$

s is the sign bit, c is the stored exponent (characteristic), and f is the fractional part of the mantissa.

Not every real number can be represented in this form. If a computed number is too large to fit within this range, it results in an *overflow* error. Similarly, if a computed number is too small in magnitude to be represented as a normalized number, it results in an *underflow*. Underflow numbers are typically set to zero.

Finally, let us relate this representation to the normalized scientific notation introduced earlier. Recall that we wrote a binary number as

$$x_2 = \pm q_2 \times 2^m,$$

where

$$0.1_2 \leq q_2 < 1.0_2.$$

In the IEEE representation,

$$q_2 = \frac{1+f}{2} \quad \text{and} \quad m = c - 1022.$$

Thus, the IEEE floating-point format is simply a carefully designed way of storing the mantissa q_2 and exponent m in binary, while balancing range, precision, and computational efficiency.

1.2 Errors

We have discussed the conversion of numbers and the errors it can produce. Now, we will concentrate on the decimal number system. Since real-life numbers can have infinitely many digits, but a computer has finite storage, errors can occur while storing numbers.

The errors produced when a calculator or a computer is used to perform real-number calculations are called *round-off errors*. Any positive real number within the numerical range of a machine can be represented as

$$y = 0.d_1 d_2 \dots d_k d_{k+1} \cdots \times 10^n.$$

The floating-point representation of y, denoted by $f_\ell(y)$, is obtained by terminating the mantissa of y after k decimal digits.

This termination can happen in two ways:

1. **Chopping**: In this method, we simply chop off all digits after the k-th digit, that is, the digits $d_{k+1}, d_{k+2}, \dots$ are discarded.

2. **Rounding**: In this method, if $d_{k+1} < 5$, we keep

$$f_\ell(y) = 0.d_1 d_2 \dots d_k \times 10^n.$$

If $d_{k+1} \geq 5$, then we increase the k-th digit by one and write

$$f_\ell(y) = 0.d_1 d_2 \dots (d_k + 1) \times 10^n.$$

For example, let us take the number π. We have

$$\pi = 0.3141592 \cdots \times 10^1.$$

If we use five-digit floating-point arithmetic, then:

- using chopping,

$$f_\ell(\pi) = 0.31415 \times 10^1,$$

- using rounding,

$$f_\ell(\pi) = 0.31416 \times 10^1.$$

1.2.1 Measure of Error

After understanding the different kinds of errors, we now look at how to *measure* these errors.

Definition 1.4. Let p^* be an approximation of p. The *absolute error* is defined as

$$|p - p^*|,$$

and the *relative error* is defined as

$$\frac{|p - p^*|}{|p|},$$

provided that $p \neq 0$.

If the quantity p has physical dimensions (for example, meters or kilograms), then the absolute error has the same dimension, whereas the relative error is dimensionless. Both types of error have their advantages and disadvantages.

Let us first look at relative error. Suppose we have two wooden rods of lengths 1 m and 100 m, and we measure their lengths as 2 m and 101 m, respectively. In both cases, the absolute error is 1 m. However, the relative errors are

$$\frac{1}{1} = 100\% \quad \text{and} \quad \frac{1}{100} = 1\%,$$

respectively. This shows that absolute error alone can be misleading.

On the other hand, if the quantity being approximated is very close to zero, the relative error may become extremely large or even meaningless. In such cases, the absolute error is more informative.

We now introduce the concept of significant digits.

Definition 1.5. The number p^* is said to approximate p to t *significant digits* if t is the largest non-negative integer such that

$$\frac{|p - p^*|}{|p|} \leq 5 \times 10^{-t}.$$

Example 1.6. Suppose we approximate 0.5 by 0.501. Then the relative error is

$$\frac{|0.5 - 0.501|}{0.5} = \frac{0.001}{0.5} = 0.002.$$

We now determine the largest integer t such that

$$0.002 \leq 5 \times 10^{-t}.$$

This gives

$$\begin{aligned} 10^{-t} &\geq 4 \times 10^{-4}, \\ t &\leq \log_{10}(2500), \\ t &\approx 3. \end{aligned}$$

Hence, the approximation is correct up to three significant digits.

1.2.2 Arithmetic Errors

Apart from round-off errors, we may also encounter errors while performing arithmetic operations. Let x and y be two real numbers whose floating-point representations are denoted by $f_\ell(x)$ and $f_\ell(y)$. Let $\oplus$, $\ominus$, $\otimes$, and $\oslash$ denote machine addition, subtraction, multiplication, and

division, respectively. Then finite-point arithmetic is defined as

$$\begin{aligned} x \oplus y &= f_\ell\left(f_\ell(x) + f_\ell(y)\right), \\ x \ominus y &= f_\ell\left(f_\ell(x) - f_\ell(y)\right), \\ x \otimes y &= f_\ell\left(f_\ell(x) \times f_\ell(y)\right), \\ x \oslash y &= f_\ell\left(f_\ell(x) \div f_\ell(y)\right). \end{aligned}$$

Example 1.7. Let

$$x = \frac{5}{7}, \qquad y = \frac{1}{3}.$$

Suppose we use five-digit floating-point arithmetic with chopping.
First, we write both numbers in normalized decimal scientific notation and apply chopping:

$$x = 0.71428 = 0.71428 \times 10^0,$$

$$y = 0.33333\ldots = 0.33333 \times 10^0.$$

Hence,

$$f_\ell(x) = 0.71428 \times 10^0, \qquad f_\ell(y) = 0.33333 \times 10^0.$$

Now we compute

$$\begin{aligned} x \oplus y &= f_\ell\left((0.71428 + 0.33333) \times 10^0\right) \\ &= f_\ell(1.04761 \times 10^0) \\ &= 0.10476 \times 10^1. \end{aligned}$$

The exact value is

$$x + y = \frac{5}{7} + \frac{1}{3} = \frac{22}{21} = 1.04761904\ldots$$

Hence, the absolute error is

$$|(x+y) - (x \oplus y)| \approx 0.19 \times 10^{-4},$$

and the relative error is

$$\frac{|(x+y) - (x \oplus y)|}{|x+y|} \approx 0.182 \times 10^{-4}.$$

The reason we study such errors is that due to finite-point arithmetic, computers may accumulate numerical errors. If an algorithm is applied recursively, a small error at an early stage can grow rapidly and lead to large inaccuracies later. This phenomenon is referred to as *numerical instability*.

Example 1.8. Consider the sequence of real numbers defined by

$$\begin{aligned} x_0^{\mathrm{R}} &= 1, \qquad x_1^{\mathrm{R}} = \frac{1}{3}, \\ x_{n+1}^{\mathrm{R}} &= \frac{13}{3} x_n^{\mathrm{R}} - \frac{4}{3} x_{n-1},^{\mathrm{R}} \qquad (n \geq 1). \end{aligned}$$

It is easy to verify that the exact solution of this recurrence relation is

$$x_n^{\mathrm{T}} = \left(\frac{1}{3}\right)^n.$$

Now, let us generate the sequence numerically using the recurrence relation and compare it with the exact values and setting the rounding to 7 significant digits. Table 1.2 shows the results for the first 15 values of n. We observe that as n increases, the error also increases. Hence, the algorithm is numerically unstable: a small perturbation in the initial values leads to large errors in later iterations.

n	x_n^{R}	x_n^{T}	Error
0	1.0000000	1.0000000	0.0000000
1	0.3333333	0.3333333	0.0000000
2	0.1111110	0.1111111	0.0000001
3	0.0370366	0.0370370	0.0000004
4	0.0123439	0.0123457	0.0000018
5	0.0041081	0.0041152	0.0000071
6	0.0013432	0.0013717	0.0000285
7	0.0003431	0.0004572	0.0001141
8	-0.0003042	0.0001524	0.0004566
9	-0.0017757	0.0000508	0.0018265
10	-0.0072891	0.0000169	0.0073060
11	-0.0292185	0.0000056	0.0292241
12	-0.1168947	0.0000019	0.1168966
13	-0.4675857	0.0000006	0.4675863
14	-1.8703451	0.0000002	1.8703453
15	-7.4813812	0.0000001	7.4813813

Table 1.2: Computed values, exact values, and absolute error

The above example motivates the concept of *conditioning*, which we will discuss later when solving systems of equations. Informally speaking, conditioning measures how sensitive the output of a problem is to small changes in the input.

Until now, the errors we have encountered are mainly due to the computer itself: how numbers are stored and how arithmetic operations are performed on them. However, errors can also arise due to the approximation of mathematical expressions.

Let us consider the Taylor series expansion of a function $f(x) \in \mathcal{C}^\infty[a, b]$ about the

point x. Then,

$$f(x+h) = f(x) + hf'(x) + \frac{h^2}{2!}f^{(2)}(x) + \cdots + \frac{h^n}{n!}f^{(n)}(x) + \cdots .$$

As we will learn later, the Taylor series is a powerful tool that allows us to construct numerical methods by approximating a function using only the first few terms of its expansion. However, since we are working with finite memory and finite computational resources, we must truncate the Taylor series after a finite number of terms.

The error introduced by neglecting the remaining terms of the series is called the *truncation error*. This topic will be revisited in detail when we study the numerical solution of ordinary and partial differential equations.

List of Notation

Notation	**Description**
$\sum_{n=1}^{\infty}$	Summation of Numbers
x_{10}	Base 10 Number
x_2	Base 2 Number
$\mathbb{Z}$	Set of Integers
$\mathcal{C}[a,b]$	Set of continuous functions defined on $[a.b]$
$\mathcal{C}^{\infty}[a,b]$	Set of infinitely differentiable functions defined on $[a.b]$

Table 1.3: List of Mathematical Notations

List of Algorithms

Index

Absolute Error, 13

Binary Number System, 9

Characteristic, 11
Chopping, 13
Conditioning, 16

Decimal Number System, 9

Exponent, 11

Mantissa, 11

Numerical Instability, 15

Overflow, 12

Relative Error, 13
Round-off Errors, 13
Rounding, 13

Significant Digits, 14

Truncation Error, 17

Underflow, 12

Bibliography

[1] "IEEE Standard for Binary Floating-Point Arithmetic". In: *ANSI/IEEE Std 754-1985* (1985), pp. 1–20. DOI: 10.1109/IEEESTD.1985.82928.

[2] "IEEE Standard for Floating-Point Arithmetic". In: *IEEE Std 754-2008* (2008), pp. 1–70. DOI: 10.1109/IEEESTD.2008.4610935.